

Travel & Tourism ML Capstone

Productionization & MLOps Documentation

Flight Price Prediction Service & Hotel Recommendation App

Covers: REST API (Flask) · Streamlit App · Docker · Kubernetes · Apache Airflow · Jenkins CI/CD · MLflow

Table of Contents

- 0. Introduction
- 1. REST API (Flask) — Flight Price Prediction
- 2. Streamlit App — Hotel Recommendation
- 3. Docker Deployment
- 4. Kubernetes Deployment
- 5. Scheduling with Apache Airflow
- 6. CI/CD Pipeline (Jenkins)
- 7. MLflow Tracking
- 8. Cross-Cutting Design Principles
- 9. Project Structure
- 10. Summary — What Has Been Built

0. Introduction

This document describes the productionization layer built on top of the flight ticket price prediction model and the hybrid hotel recommendation model developed for this capstone. Where the modeling notebooks (Flight_Ticket_Price_Prediction.ipynb, Gender_classification.ipynb, and Hotel_Recommendation.ipynb) cover data preparation, feature engineering, and model training, this document covers everything built to take the flight price model from a saved artifact to a running, served, containerized, orchestrated, and monitored system — and to expose the hotel recommendation model through an interactive application.

Seven components are documented in the sections that follow, each backed by a working, runnable implementation and demonstrated with screenshots taken from the actual running services:

- A Flask REST API that serves real-time flight price predictions.
- A Streamlit application that serves hybrid hotel recommendations.
- A Docker image that packages the Flask app for portable, repeatable deployment.
- A Kubernetes deployment that runs the containerized app at scale with autoscaling.
- An Apache Airflow DAG that automates scheduled batch scoring of the flight price model.
- A Jenkins pipeline that gates every code change behind automated tests before it is built and deployed.
- An MLflow tracking setup that logs and compares every model training run.

Together, these seven pieces form a complete, working MLOps loop: a change to the model or application code is tested, containerized, deployed, and continuously scored, with every training run traceable back to its metrics and parameters.

0.1 System Overview

At a high level, the flight price prediction system flows as follows: the flight dataset is used to train and compare candidate regression models, with every run tracked in MLflow; the best-performing model (a tuned XGBoost regressor) is exported as `best_xgb_model.pkl`, alongside its fitted `encoder.pkl` and `scaler.pkl`; a Flask application (`app.py` + `predict.py`) loads these three artifacts and exposes a form-based prediction interface; this application is packaged into a Docker image; the image is deployed to Kubernetes as a scalable, self-healing set of pods behind a load-balanced service; an Airflow DAG periodically invokes the same prediction logic in batch, scoring every known route combination and appending results to a running predictions file; and a Jenkins pipeline ties the whole delivery process together, running tests on every change and — when explicitly enabled — pushing new images and rolling out new deployments.

Separately, the hotel recommendation model (a hybrid of collaborative filtering and content-based filtering, trained in the Hotel Recommendation notebook) is exposed through its own Streamlit application, chosen because the intended interaction — browsing recommendations, filtering hotels, and reviewing booking history — is exploratory and multi-panel rather than a single request/response exchange.

0.2 Component Map

The table below maps each artifact produced during modeling to the specific production component that consumes it, so it is clear how the pieces documented in this report connect back to the notebooks:

Artifact	Produced By	Consumed By
<code>best_xgb_model.pkl</code> / <code>encoder.pkl</code> / <code>scaler.pkl</code>	<code>Flight_Ticket_Price_Prediction.ipynb</code>	<code>predict.py</code> , called by both the Flask app (Section 1) and the Airflow batch DAG (Section 5)

load_distance_time.csv	Flight_Ticket_Price_Prediction.ipynb (route-level aggregation)	predict.py — fills in time/distance for a request that only supplies route, date, flight type, and agency
collaborative_recommendation.pkl (SVD model)	Hotel_Recommendation.ipynb	The Streamlit app's hybrid recommendation logic (Section 2)
encoded_features_scaled (content feature matrix)	Hotel_Recommendation.ipynb	The Streamlit app's content-based cosine-similarity lookup (Section 2)
pricepred-app Docker image	Section 3 (Docker)	Section 4 (Kubernetes) and Section 6 (Jenkins Docker Push / Deploy stages)
mlruns/ tracking store	flight_ticket_price_prediction.py training runs	MLflow Tracking UI (Section 7) and the MLflow Model Registry entry best_model

0.3 Environment & Tooling

All seven components were run and verified locally, using the following tools, so that the entire pipeline could be demonstrated end-to-end without depending on any paid cloud account:

- Docker Desktop, with its built-in single-node Kubernetes cluster (minikube) used for the Kubernetes deployment in Section 4.
- A local Jenkins instance (<http://localhost:8081>) running the pipeline described in Section 6.
- A local Apache Airflow instance, itself running inside Docker Compose (`infra/airflow/`) with a Postgres metadata database, as described in Section 5.
- A local MLflow tracking server (<http://127.0.0.1:5500>) backed by a file-based mlruns/ store, as described in Section 7.

Because every manifest, Dockerfile, and pipeline definition here is written against standard, portable interfaces (Kubernetes' API, Docker's image format, Jenkins' declarative pipeline syntax), the same definitions apply unmodified against managed equivalents — this is called out specifically in Section 4 for Kubernetes, where it matters most.

0.4 Glossary of Key Terms

A short reference for terms used throughout the sections that follow:

Term	Meaning in this document
Encoder / Scaler	The fitted OneHotEncoder (for categorical fields) and MinMaxScaler (for numeric fields) saved during training and reused, unchanged, at serving time.
Pickled model (.pkl)	A serialized Python object — here, a trained scikit-learn/XGBoost model, encoder, or scaler — saved to disk with Python's pickle format so it can be loaded later without retraining.
Container / Image	A Docker image is the packaged, runnable snapshot of the app and its dependencies; a container is a running instance of that image.
Pod / Replica	In Kubernetes, a pod is one running instance of a container; a Deployment's replica count is how many identical pods run at once.

HPA	HorizontalPodAutoscaler — a Kubernetes controller that changes a Deployment's replica count automatically based on an observed metric such as CPU utilization.
DAG	In Airflow, a Directed Acyclic Graph — the definition of a workflow's tasks and the order they run in. This project's DAG has a single task.
Upsert	Update-or-insert: for each new row, update it if a matching row already exists, otherwise insert it — used here so batch predictions accumulate rather than overwrite history.
Pipeline (Jenkins)	A defined sequence of stages (checkout, test, build, push, deploy) that Jenkins runs automatically in response to a trigger such as a code push.
Smoke test	A fast, narrow test that checks whether the most basic functionality works at all — here, that <code>predict()</code> returns a sane number and that the app's root route responds.
Run (MLflow)	One tracked execution of a training/evaluation script, with its own logged parameters, metrics, and model artifact, comparable against every other run.
Model Registry	MLflow's system for giving a specific run's model artifact a stable, versioned name (here, <code>best_model</code>) independent of the underlying run ID.
SVD	Singular Value Decomposition — the matrix-factorization technique behind the collaborative-filtering half of the hotel recommender, used to predict a user's rating for hotels they have not booked.
Cosine similarity	A measure of how similar two feature vectors are by the angle between them — used in the content-based half of the hotel recommender to find hotels similar to a user's own feature profile.
Cold start	The case where a recommender has no history for a given user (or item) to base a personalized recommendation on — handled here by falling back to overall popularity.

1. REST API (Flask) — Flight Price Prediction

Endpoint (local): `http://localhost:5000`

1.1 Purpose

The flight price model needs to be reachable by something other than a Python notebook: a user filling in trip details and expecting an instant price estimate. Flask was chosen as the serving framework because the interaction is a simple, synchronous request/response cycle — submit trip details, receive one predicted price — which maps directly onto a single form-post route without needing the overhead of a larger web framework.

1.2 Application Structure

The application is organized around three concerns, kept deliberately separate so that the model logic can be tested, reused, and updated independently of the web layer:

- `app.py` — the Flask entry point. Its `home()` route renders an empty input form on a GET request, and on a POST request collects the submitted fields into a one-row pandas DataFrame before handing off to the prediction logic.
- `predict.py` — holds the `preprocess()` and `predict()` functions built in the modeling notebook, plus a `batch_predictions()` function (reused later by the Airflow DAG).

- `pkl_files/` — the serialized `encoder.pkl`, `scaler.pkl`, and `best_xgb_model.pkl` produced by the training notebook.
- `templates/` — the HTML template rendered by Flask, containing the input form and the result banner.

1.3 Request/Response Flow

The end-to-end flow for a single prediction request is:

- The form collects four fields from the user: Date, From, To, Flight Type, and Agency. Deliberately, flight time and distance are not asked for — they are not something a user booking a ticket would know in advance.
- On submission, `app.py` assembles these fields into a one-row `DataFrame` and calls `predict.py`'s `predict()` function.
- Because time and distance are missing from the request, `predict()` first merges the input row against a precomputed lookup table (`load_distance_time.csv`) — built during training from the average time and distance observed for each historical `from/to/flighttype/agency` combination — to fill those two fields in.
- The categorical fields (`from`, `to`, `flighttype`, `agency`) are transformed using the saved `encoder.pkl`, and the numeric fields (including the derived date features and the looked-up time/distance) are transformed using the saved `scaler.pkl` — the exact same transformation objects fitted during training.
- The resulting feature vector is passed to the tuned XGBoost model (`best_xgb_model.pkl`), and the predicted price is rendered back into the page.

This design means the serving path reuses training-time preprocessing exactly, rather than re-implementing it — the single largest source of train/serve mismatch bugs in deployed ML systems is avoided by construction.

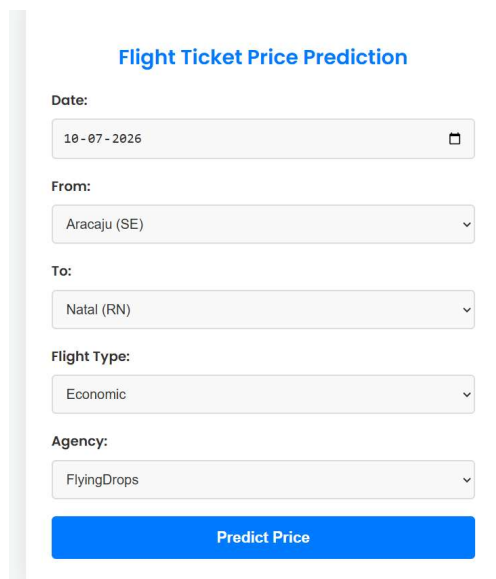
A web form titled "Flight Ticket Price Prediction" in blue. It contains five input fields: "Date:" with a text box showing "10-07-2026" and a calendar icon; "From:" with a dropdown menu showing "Aracaju (SE)"; "To:" with a dropdown menu showing "Natal (RN)"; "Flight Type:" with a dropdown menu showing "Economic"; and "Agency:" with a dropdown menu showing "FlyingDrops". At the bottom is a blue button labeled "Predict Price".

Figure 1.1 — Flight price prediction form (date, route, flight type, agency).

Predicted Price: \$1333.634

Figure 1.2 — Predicted price rendered back into the page after submission.

1.4 Why This Design

Keeping the lookup-table merge, encoding, and scaling inside `predict.py` rather than `app.py` means the exact same function that was validated in the training notebook is what runs in production, and the same function is reused

unmodified by the Airflow batch job described in Section 5 — there is only one implementation of "how a raw request becomes a prediction" in the whole system.

1.5 Input Handling

Because the form's From, To, Flight Type, and Agency fields are populated from dropdowns built off the same category values seen during training, the request path avoids the most common failure mode for a categorical encoder in production — an unseen category reaching the model. The saved encoder.pkl was fit with `handle_unknown='ignore'`, so even if a value outside the training vocabulary were ever submitted, the pipeline degrades gracefully (encoding it as all-zeros for that field) rather than raising an exception and returning an error page to the user.

1.6 Local Verification

Before being containerized, the Flask app was run and exercised directly on the host (`python app.py`, or `flask run`), confirming the full route — form render, form submission, lookup-table merge, encode, scale, predict, and render — end to end. This local run is also what the two Jenkins smoke tests described in Section 6 exercise automatically on every commit: one checks `predict()` directly against a real route, and the other checks that the Flask app's root route responds with HTTP 200.

2. Streamlit App — Hotel Recommendation

2.1 Purpose

The hotel recommendation model needs an interface that supports browsing and comparison, not just a single answer — a user should be able to see their recommendations, explore alternatives, and check their own booking history. Streamlit was chosen specifically for this reason: the interaction is exploratory and multi-panel, which fits Streamlit's dashboard-style layout far better than a single request/response Flask page would.

2.2 Hybrid Recommendation Logic Behind the App

The app is a thin interface over the hybrid recommender built in the modeling notebook, combining two independent strategies:

- Collaborative filtering — an SVD model (`collaborative_recommendation.pkl`), trained on the user–hotel interaction matrix, predicts a rating for every hotel a given user has not yet booked and returns the top-scoring ones.
- Content-based filtering — cosine similarity is computed between a user's own feature vector (derived from their booking history: price range, place, company, age group, and more, one-hot encoded and scaled) and every hotel's feature vector, surfacing hotels similar to what that user already favors.
- The two result sets are merged, with duplicates removed, to produce the final recommendation list shown in the app.
- For a "New User" with no booking history — a genuine cold-start case — the app falls back to showing the 5 most-booked hotels overall, so the interface is never left empty.

2.3 Application Panels

The app is organized into three panels, each addressing a different way a user might want to interact with the recommendation system:

2.3.1 Get Recommendations

A dropdown lets the user select their User Code; clicking Get Recommendations runs the hybrid logic live and displays both the personalized recommended hotel list and, for comparison, the top 5 most popular hotels overall.

Hybrid Hotel Recommendation System

Select User

2

Get Recommendations

Recommended Hotels:

```
[
  0 : "Hotel A"
  1 : "Hotel K"
  2 : "Hotel BD"
  3 : "Hotel Z"
  4 : "Hotel AU"
]
```

Top 5 Popular Hotels:

```
[
  0 : "Hotel K"
  1 : "Hotel CB"
  2 : "Hotel BD"
  3 : "Hotel AF"
  4 : "Hotel AU"
]
```

Figure 2.1 — Hybrid Hotel Recommendation System: recommended hotels for a selected user, alongside the overall top 5 popular hotels.

2.3.2 Explore More Hotels

A second panel lets the user filter hotels by place and by a price-per-day range using a slider, returning a live-filtered table of matching hotels — useful for a user who wants to browse beyond their personalized list.

Explore More Hotels

Filter and check hotel prices by place per day

Select Place

Salvador (BH)

Select Price Range

59 314

	hotel_name	place	price
1	Hotel K	Salvador (BH)	263.41

Figure 2.2 — Explore More Hotels: filtering hotels by place and price range.

2.3.3 Check Bookings History

A third panel displays the selected user's full booking history in a searchable, sortable table, alongside a bar chart of their total spending broken down by hotel — giving the user (or an analyst) immediate visibility into the data the recommendations are actually based on.

Check bookings history

Booking History

	date	hotel_name	place
29	2019-10-31 00:00:00	Hotel CB	Rio
30	2019-07-11 00:00:00	Hotel BW	Can
31	2019-05-12 00:00:00	Hotel Z	Ara
32	2019-12-26 00:00:00	Hotel AU	Rec
33	2020-02-01 00:00:00	Hotel BW	Can
34	2020-09-01 00:00:00	Hotel K	Salv
35	2020-01-23 00:00:00	Hotel AF	Sao
36	2020-02-20 00:00:00	Hotel BW	Can
37	2020-02-27 00:00:00	Hotel AU	Rec
38	2020-12-03 00:00:00	Hotel K	Salv

Spending by Hotel

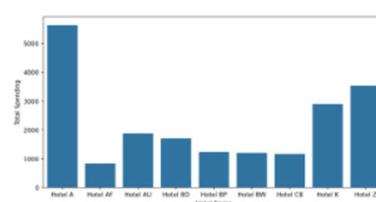


Figure 2.3 — Booking history table and spending-by-hotel chart for the selected user.

2.4 Implementation Notes

The trained SVD model is loaded once at app startup with `joblib.load()`, rather than being retrained or refit on every interaction — the app treats `collaborative_recommendation.pkl` and the scaled content-feature matrix purely as read-only artifacts, so opening the app or switching between users is instant rather than re-running any model fitting. The three panels described above all read from the same in-memory dataframes and models, so selecting a different user in the first panel and then jumping to the booking-history panel reflects that same user without any extra lookups.

3. Docker Deployment

3.1 Purpose

Packaging the Flask app in Docker removes any dependency on the specific machine it was developed on — the same image runs identically on a laptop, a teammate's machine, a Kubernetes node, or a cloud VM, eliminating environment drift between development and deployment.

3.2 Dockerfile Design

The image is built from a `python:3.10-slim-buster` base to keep the image small while still matching the Python version used during model training. The key instructions are:

Instruction	Purpose
<code>WORKDIR /app</code>	Sets <code>/app</code> as the working directory inside the container for all subsequent instructions.
<code>COPY . /app</code>	Copies the Flask application code (<code>app.py</code> , <code>predict.py</code> , etc.) into the image.

COPY ./pk1_files /app/pk1_files	Explicitly copies the trained model artifacts (encoder, scaler, model) into the image.
COPY ./templates /app/templates	Explicitly copies the HTML templates used to render the form and results.
RUN pip install --no-cache-dir -r requirements.txt	Installs exact Python dependencies with no local package cache retained, keeping the image lean.
EXPOSE 5000	Documents that the container listens on port 5000.
ENTRYPOINT: flask run --host=0.0.0.0 --port=5000	Binds Flask to all interfaces inside the container so it is reachable from outside — binding to localhost only would make the app unreachable once containerized.

3.3 Build & Run

The image is built and run with:

```
docker build -t pricepred-app .
docker run -p 5000:5000 pricepred-app
```

The build copies in the application code and pk1_files/ and templates/ folders explicitly, installs dependencies from requirements.txt, and exposes port 5000, producing a self-contained, reproducible image.

```
(mlops-travel) PS G:\projects\MLOps-Integrated-Travel\Productionization_flight_price_prediction> docker build -t pricepred-app .
[+] Building 0.4s (11/11) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 746B                               0.0s
=> [internal] load metadata for docker.io/library/python:3.10-slim-b 0.0s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                     0.0s
=> [1/6] FROM docker.io/library/python:3.10-slim-buster          0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 9.63kB                                 0.0s
=> CACHED [2/6] WORKDIR /app                                       0.0s
=> CACHED [3/6] COPY . /app                                        0.0s
=> CACHED [4/6] COPY ./pk1_files /app/pk1_files                  0.0s
=> CACHED [5/6] COPY ./templates /app/templates                  0.0s
=> CACHED [6/6] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> exporting to image                                             0.0s
=> => exporting layers                                           0.0s
=> => writing image sha256:e39ecd9d919b76b0e3a9c60a4c53adcbb94ac3ab6 0.0s
=> => naming to docker.io/library/pricepred-app                  0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/nlfmdn4fstvcbfzcfv69x5ml
```

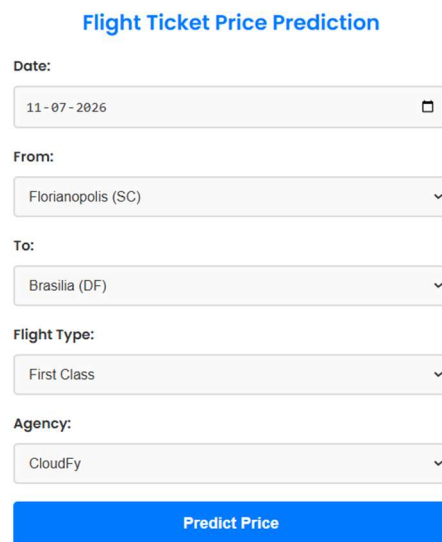
Figure 3.1 — docker build output: an 11-layer build completing in 0.4s, with dependency and file-copy layers cached.

☐	Name	Tag	Status	Created	Size	Actions
☐	pricepred-app	latest	In use	5 minutes ago	1.35 GB	▶ ⋮ 🗑
	e39ecd9d919b					

Figure 3.2 — The built pricepred-app image, tagged latest, shown as "In use" in Docker Desktop.

3.4 Verifying Parity with the Local App

To confirm the containerized version behaves identically to the version run directly on the host, the same form was submitted against the container-hosted app at localhost:5000, using a different route and flight type than the earlier local test:



Flight Ticket Price Prediction

Date:

From:

To:

Flight Type:

Agency:

Predict Price

Figure 3.3 — The same prediction form served from inside the running container.

Predicted Price: \$1284.0758

Figure 3.4 — Predicted price returned by the containerized instance, confirming behavior matches the local run.

This confirms the containerized app is functionally identical to the one run directly on the host — the same code path, the same artifacts, and the same output, just reachable through a portable image rather than a local Python process.

3.5 Image Composition

The build log in Figure 3.1 shows the layer structure that results from this Dockerfile: the base Python image and the pip install step are the largest, slowest-changing layers, while the application code, `pkl_files/`, and `templates/` copies are kept as separate, later layers. This ordering matters for build speed — because `requirements.txt` changes far less often than `app.py` during development, Docker can reuse ("CACHED") the dependency-installation layer across most rebuilds, as seen directly in Figure 3.1 where five of the six meaningful build steps are served from cache and only the final export step does new work. Docker Desktop's image list (Figure 3.2) confirms the resulting image — tagged `pricepred-app:latest` — at roughly 1.35 GB, dominated by the Python base image and the scientific-computing dependencies (pandas, scikit-learn, XGBoost) rather than by the application code itself, which is only a few kilobytes.

4. Kubernetes Deployment

4.1 Purpose

A single Docker container is a single point of failure and a hard ceiling on throughput. Kubernetes is used to run the same image as a managed set of replicas, so the app can absorb traffic spikes by scaling out, recover automatically from a crashed pod, and be reached through one stable address regardless of which underlying pod answers a given request.

4.2 Manifests

The Flask app is deployed to Kubernetes using three manifests, applied together to a local minikube cluster:

Manifest	Role
deployment.yaml	Runs 3 replicas of the pricepred-app container, each listening on port 5000, with defined CPU and memory requests and limits so the scheduler can place pods appropriately and no single pod can starve the node.
service.yaml	Exposes the deployment as a LoadBalancer Service, mapping external port 80 to the container's port 5000, giving the app one stable network address regardless of which pod handles a request.
hpa.yaml	Attaches a HorizontalPodAutoscaler that scales the deployment between 1 and 10 replicas automatically, triggered when average CPU utilization crosses 50%.

4.3 Applying the Manifests

```
kubectl apply -f deployment.yaml -f service.yaml -f hpa.yaml
```

Applying all three manifests brings up 3 running pods, a stable cluster-internal service endpoint, and an active autoscaler watching CPU load — all from manifests that would work unmodified against a managed cloud cluster such as EKS, GKE, or AKS, not just a local minikube instance.



Figure 4.1 — The pricepred-app image referenced by the local Kubernetes (minikube) cluster in Docker Desktop, showing current CPU utilization.

4.4 Design Notes

Setting explicit CPU/memory requests and limits in deployment.yaml is what makes the HorizontalPodAutoscaler's CPU-percentage target meaningful — without a defined request, "50% of CPU" has no baseline to measure against. Using a LoadBalancer Service type (rather than NodePort) also means the same manifest maps directly onto a cloud provider's real load balancer with no changes required when moving from minikube to a managed cluster.

4.5 Manifest Structure

The three manifests follow the standard Kubernetes shape for a scalable web service — a Deployment describing the desired pod replicas and their resource envelope, a Service exposing them under one address, and an HPA watching utilization. In outline:

```
deployment.yaml: kind=Deployment, spec.replicas=3, container port 5000, resources.requests / resources.limits set for
cpu and memory

service.yaml: kind=Service, type=LoadBalancer, port 80 -> targetPort 5000

hpa.yaml: kind=HorizontalPodAutoscaler, minReplicas=1, maxReplicas=10, target: averageUtilization 50% (cpu)
```

4.6 Autoscaling Behavior

With this configuration, the deployment starts at 3 replicas. If average CPU utilization across the running pods climbs above 50%, the HPA increases the replica count, up to the configured ceiling of 10; if load drops back down, it scales back in, down to the configured floor of 1. Figure 4.1 shows the cluster's tracked CPU utilization for the running pricepred-app image at the time of observation — a live, non-zero figure confirming the autoscaler has an actual metric to act on, rather than the deployment simply sitting idle.

5. Scheduling with Apache Airflow

5.1 Purpose

A price prediction that only happens when a user fills in a form doesn't produce a dataset of predictions over time. The Airflow DAG exists to run the same prediction logic in batch, on a schedule, so that a historical record of predicted prices across every known route accumulates automatically, without manual intervention.

5.2 DAG Design

Batch scoring is scheduled via a single-task DAG named `flight_price_batch_predictions`, running inside a Docker Compose stack (`infra/airflow/`) alongside a Postgres metadata database, using Airflow's `LocalExecutor`.

- The DAG's single task calls `batch_predictions()` from `predict.py` — the same module used by the Flask app, so batch scoring and real-time scoring share one implementation of the prediction logic.
- `batch_predictions()` scores every known (from, to, flighttype, agency) route combination for the current date.
- Results are upserted into `predictions.csv`: new rows for the current date are merged with any existing rows, so historical predictions for other dates are preserved rather than overwritten on every run.
- The DAG is scheduled every 10 minutes for this demo (`*/10 * * * *`); in a production setting this would move to a daily cadence (`@daily`), since flight prices and route lists do not change fast enough to warrant more frequent scoring.

5.3 Resolving Earlier Design Issues

This DAG replaced an earlier, non-working attempt. The original design created a Python virtual environment in one task and then ran the actual scoring logic in a separate task using a different interpreter — meaning the virtual environment created in the first task was never actually used by the second. That version also failed with a `PermissionError` when writing output under a container bind mount. Both issues were resolved in the current DAG: scoring now runs directly within a single task, with all dependencies baked directly into the Airflow image rather than built at runtime, and the output-file write is wrapped defensively to handle the bind-mount permission case.

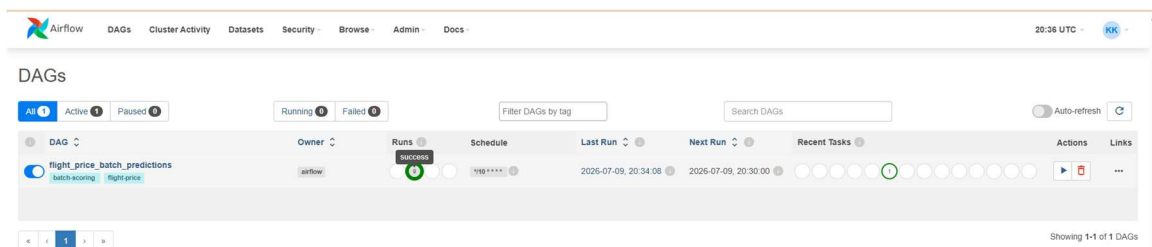


Figure 5.1 — The `flight_price_batch_predictions` DAG, active and showing a successful run.

State	Dag Id	Logical Date	Run Id	Run Type	Queued At	Start Date	End Date	Note	External Trigger	Conf	Duration
success	flight_price_batch_predictions	2026-07-09, 20:34:08	manual__2026-07-09T20:34:08+00:00	manual	2026-07-09, 20:34:08	2026-07-09, 20:34:09	2026-07-09, 20:34:24	True		0	15s
success	flight_price_batch_predictions	2026-07-09, 20:20:00	scheduled__2026-07-09T20:20:00+00:00	scheduled	2026-07-09, 20:32:08	2026-07-09, 20:32:08	2026-07-09, 20:32:30	False		0	22s
success	flight_price_batch_predictions	2026-07-09, 19:03:52	manual__2026-07-09T19:03:52+00:00	manual	2026-07-09, 19:03:52	2026-07-09, 19:03:53	2026-07-09, 19:03:58	True		0	5s
success	flight_price_batch_predictions	2026-07-09, 18:50:00	scheduled__2026-07-09T18:50:00+00:00	scheduled	2026-07-09, 19:02:29	2026-07-09, 19:02:29	2026-07-09, 19:02:39	False		0	10s
success	flight_price_batch_predictions	2026-07-09, 17:59:21	manual__2026-07-09T17:59:21+00:00	manual	2026-07-09, 17:59:21	2026-07-09, 17:59:22	2026-07-09, 17:59:29	True		0	7s
success	flight_price_batch_predictions	2026-07-09, 17:57:51	manual__2026-07-09T17:57:51+00:00	manual	2026-07-09, 17:59:12	2026-07-09, 17:59:13	2026-07-09, 19:02:39	True		0	1h.3M.26s
success	flight_price_batch_predictions	2026-07-09, 17:50:54	manual__2026-07-09T17:50:54+00:00	manual	2026-07-09, 17:59:12	2026-07-09, 17:59:13	2026-07-09, 17:59:22	True		0	9s
success	flight_price_batch_predictions	2026-07-09, 17:50:00	scheduled__2026-07-09T17:50:00+00:00	scheduled	2026-07-09, 18:00:00	2026-07-09, 18:00:00	2026-07-09, 18:00:08	False		0	7s
success	flight_price_batch_predictions	2026-07-09, 17:40:00	scheduled__2026-07-09T17:40:00+00:00	scheduled	2026-07-09, 17:59:12	2026-07-09, 17:59:13	2026-07-09, 19:02:39	False		0	1h.3M.26s

Figure 5.2 — DAG run history: alternating scheduled and manually triggered runs, all completing with a success state.

5.4 Outcome

With this DAG running, the system now maintains a continuously growing, deduplicated record of predicted flight prices across every known route, updated automatically on schedule rather than only when a user happens to submit the form — turning the Flask app's single-request logic into a standing dataset useful for downstream analysis or monitoring.

6. CI/CD Pipeline (Jenkins)

Endpoint (local): `http://localhost:8081`

6.1 Purpose

Without an automated pipeline, deploying a code change means manually rebuilding the image, manually pushing it, and manually re-applying the Kubernetes manifests — a process that is slow and easy to get wrong. The Jenkins pipeline exists to make every step from code change to deployment repeatable and gated behind passing tests, whether or not a real registry and cluster are available to push to.

6.2 Pipeline Stages

The pipeline is defined in `Productionization_flight_price_prediction/Jenkinsfile` and runs on every trigger:

Stage	What it does
Checkout	Checks out the repository at the triggering commit.
Test	Runs two pytest smoke tests from <code>tests/test_predict.py</code> : one confirms <code>predict()</code> returns a finite, positive price for a real route; the other confirms the Flask app's root route returns HTTP 200.
Docker Build	Builds the Docker image, tagging it with both the current build number and latest.
Docker Push (gated)	Pushes the built image to a registry — gated behind an explicit <code>PUSH_TO_REGISTRY</code> parameter, which defaults to false.
Kubernetes Deploy (gated)	Rolls the new image out to the Kubernetes deployment — gated behind an explicit <code>DEPLOY_TO_K8S</code> parameter, which also defaults to false.

6.3 Why the Push and Deploy Stages Are Gated

Both the Docker Push and Kubernetes Deploy stages are gated behind parameters that default to false. This means the pipeline can run fully green as a build-and-test gate on every commit without needing real Docker Hub credentials or a live cluster available on every single run. Flipping `PUSH_TO_REGISTRY` and `DEPLOY_TO_K8S` to true — with real credentials configured under Jenkins' Credentials Manager — is what actually pushes the image and rolls it out to Kubernetes. This separates "did this change break anything" (always checked) from "should this change actually ship" (an explicit decision), which is exactly the separation a CI/CD gate is meant to provide.

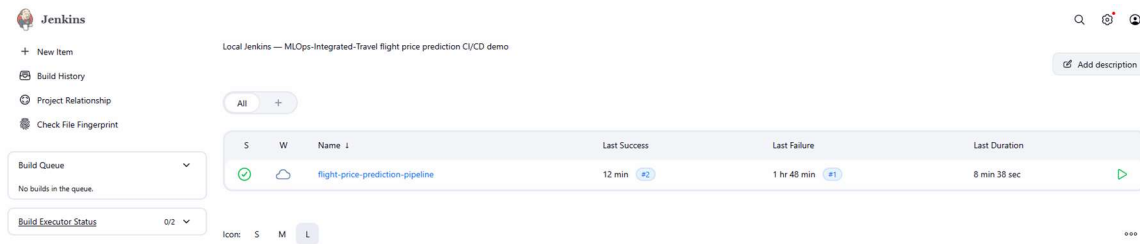


Figure 6.1 — Jenkins dashboard showing the flight-price-prediction-pipeline job, with its last successful and last failed builds.

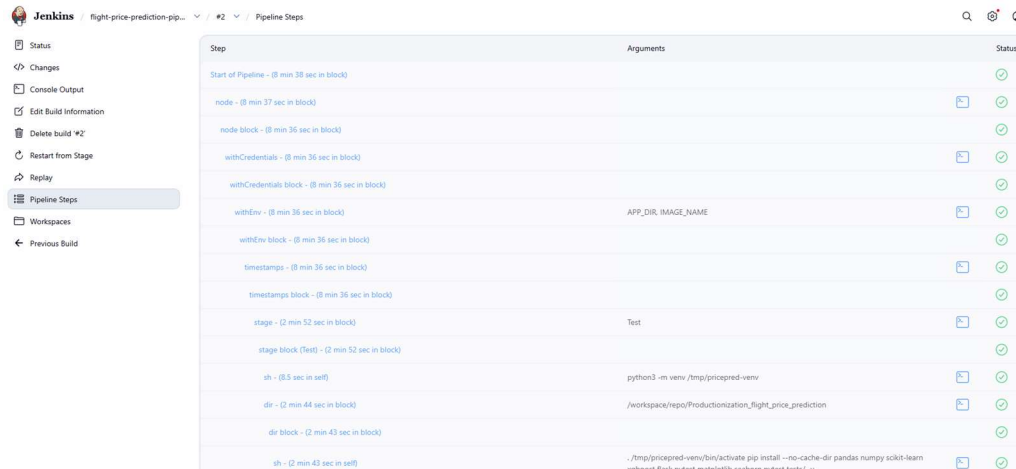


Figure 6.2 — Pipeline Steps view for a completed build: node setup, credentials/env binding, and the Test stage executing inside a virtual environment.

6.4 Test Design

Keeping the smoke tests narrow and fast — one numeric sanity check on the model's output, one HTTP status check on the web layer — means the pipeline can run on every commit in well under a minute, catching the two most common ways this kind of service breaks (a bad model output, or a broken route) without needing a slow, exhaustive test suite to gate every build.

6.5 Execution Environment

The Pipeline Steps view in Figure 6.2 shows how the Test stage is actually executed: the pipeline runs inside a node block, wraps its steps in a withCredentials block for secure credential handling, and uses a withEnv block to expose two environment variables — APP_DIR and IMAGE_NAME — to every step that follows, so the working directory and image tag are defined once rather than repeated in every shell command. Timestamps are enabled for the whole run (the timestamps/timestamps block wrapping everything), giving a precise per-step duration for build diagnostics.

Within the Test stage itself, the pipeline creates an isolated Python virtual environment (python3 -m venv /tmp/pricepred-venv) local to the Jenkins workspace, activates it, and installs the project's runtime and test dependencies — pandas, numpy, scikit-learn, xgboost, flask, pytest, matplotlib, and seaborn — before running the test suite. Isolating dependencies into a fresh virtual environment on every build means the pipeline's test results are not affected by whatever else happens to be installed on the Jenkins host, and each run starts from the same known dependency set defined in requirements.txt.

7. MLflow Tracking

Tracking UI (local): <http://127.0.0.1:5500>

7.1 Purpose

Comparing regression models by re-running notebook cells and eyeballing printed metrics does not scale past one or two runs, and leaves no durable record of what was tried. MLflow was used to track every model training run from `flight_ticket_price_prediction.py` to a local file-based store (`mlruns/`), so every run's parameters and metrics are logged, searchable, and comparable side by side.

7.2 Tracking Infrastructure

Each training run wraps its model-fitting and evaluation code in an `mlflow.start_run()` context. Inside that context, `mlflow.log_param()` records the run's configuration (for the tuned run, the winning `RandomizedSearchCV` parameters), `mlflow.log_metric()` records every evaluation metric, and `mlflow.sklearn.log_model()` logs the fitted model object itself as an artifact, so the model referenced by a run is not just described by its metrics but is actually retrievable from the run. The Runs view (Figure 7.1) also shows a Models column linking each run directly to its logged sklearn model, and supports filtering runs by a query (here, `metrics.rmse < 1` and `params.model = "tree"`) — useful once more than a handful of runs accumulate.

7.3 Runs Logged

Four runs were logged for this comparison, all evaluated on identical train/validation splits:

Run	R ²	RMSE	Duration
LinearRegression	0.9123	104.401	5.6s
RandomForest	0.999997	0.5608	22.1s
XGBoost (default)	0.999982	1.5132	4.3s
XGBoost Hyperparameter Tuning	0.9999999871	0.0400	33.7s

Run Name	Created	Dataset	Duration	Source	Models	r2	rmse	time_taken
XGBoost Hyperparameter Tuning	1 year ago	-	33.7s	flight_ticket_price_prediction.py	sklearn	0.9999999871	0.040024163...	-
XGBoost	1 year ago	-	4.3s	flight_ticket_price_prediction.py	sklearn	0.999981580...	1.513228601...	0.482559919...
RandomForest	1 year ago	-	22.1s	flight_ticket_price_prediction.py	sklearn	0.999997472...	0.560578531...	17.95232987...
LinearRegression	1 year ago	-	5.6s	flight_ticket_price_prediction.py	sklearn	0.912324698...	104.4012830...	0.111802577...

Figure 7.1 — MLflow Runs view: all four tracked runs with their `r2`, `rmse`, and `time_taken` metrics, filtered to tree-based models with `rmse < 1`.

7.4 Model Selection Reasoning

Random Forest achieved a marginally better RMSE than the default XGBoost model, but took roughly 38 times longer to train (about 22 seconds versus well under a second). Given that difference in training cost for a negligible accuracy gain, XGBoost was carried forward into a RandomizedSearchCV hyperparameter search, logged as a fourth run named "XGBoost Hyperparameter Tuning." That run's best parameters and final metrics — adjusted_r2, explained_variance, mse, r2, and rmse — were logged to MLflow, and its resulting model artifact is what the Flask app and Airflow DAG both use for serving.

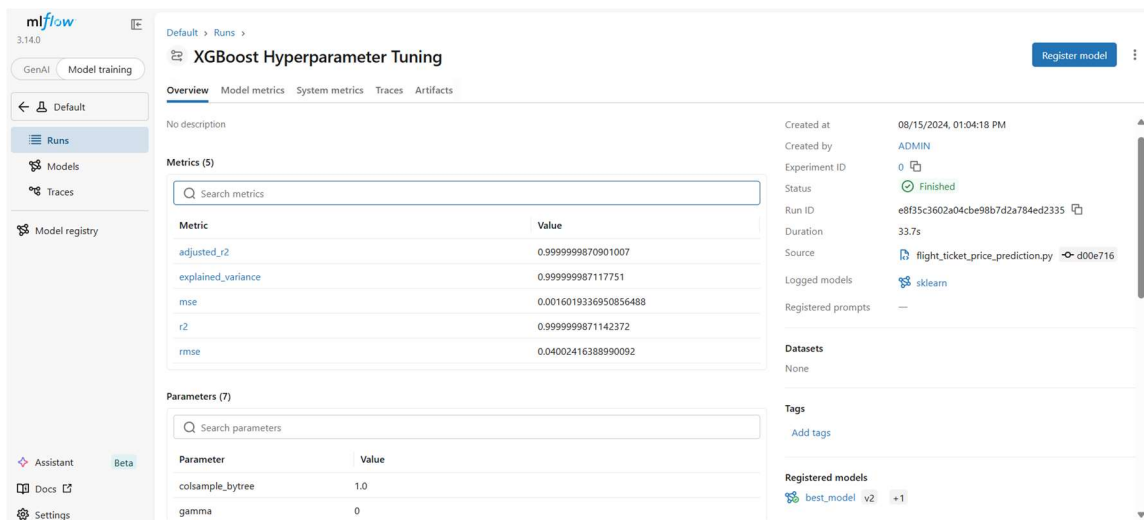


Figure 7.2 — Detail view of the XGBoost Hyperparameter Tuning run: its five logged metrics, tuned parameters, and its registration as `best_model (v2)` in the MLflow Model Registry.

7.5 Full Metrics — Winning Run

The complete metric set logged for the XGBoost Hyperparameter Tuning run, as shown in Figure 7.2:

Metric	Value
adjusted_r2	0.9999999870901007
r2	0.9999999871142372
rmse	0.04002416388990092
mse	0.0016019336950856488
explained_variance	0.999999987117751

This run is also the one registered into the MLflow Model Registry as `best_model (version 2)`, giving it a versioned, named identity distinct from the raw run ID — visible in the "Registered models" panel in Figure 7.2.

7.6 An Honest Observation on Model Performance

One result is worth flagging directly rather than glossing over: R^2 values across the three tree-based/linear comparisons range from about 0.91 up to roughly 0.9999999871 for the tuned model — unusually high for a price regression task. This is treated here as a possible data-leakage signal rather than a purely positive result. The likely source is time and distance, which are merged into every prediction from a precomputed per-route lookup table and may be too strongly correlated with price within this particular dataset, since routes with distinctive time/distance combinations may map almost one-to-one to a narrow price band.

It is also worth noting precisely how tracking and serving currently relate to each other in this system: MLflow tracks the training runs and has a model registered under `best_model (v2)` in its Model Registry, but the deployed Flask app loads its model from a plain pickled file (`best_xgb_model.pkl`) rather than pulling from that registry at serving time. In other words, the artifact MLflow tracks and the artifact currently being served are the same model, exported through two different paths — the registry entry documents and versions it, while the pickle file is what the running application actually loads.

7.7 Outcome

With this tracking in place, every candidate model that was trained for this project — its metrics, its parameters, and its training time — is preserved and comparable in one place, and the decision to promote XGBoost over Random Forest, and the tuned XGBoost run over the default one, is traceable back to the exact numbers that justified it rather than a remembered impression from re-running a notebook cell.

8. Cross-Cutting Design Principles

A few design choices recur across more than one of the seven components above. Calling them out together here makes clear that they were deliberate decisions applied consistently, rather than incidental to any single piece.

8.1 One Implementation of the Prediction Logic

`predict.py`'s `preprocess()` and `predict()` functions are the single place where "raw trip details become a price" is implemented. The Flask app calls them for a single real-time request; the Airflow DAG calls the same functions, through `batch_predictions()`, for every known route on a schedule. There is no second, parallel implementation of the encoding/scaling/inference steps anywhere in the system, so a fix or change made in one place is automatically correct in both the real-time and batch paths.

8.2 Reusing Saved Training-Time Artifacts

The encoder, scaler, and model are all fitted exactly once, during training, and saved to disk. Every downstream consumer — Flask, the Airflow DAG, and by extension the Docker image and Kubernetes pods that run Flask — loads those same saved objects rather than re-fitting anything. This guarantees that a categorical value is encoded the same way, and a numeric feature is scaled the same way, regardless of which part of the system produced the request.

8.3 Portability Through Standard Interfaces

The Docker image, the Kubernetes manifests, and the Jenkins pipeline are all written against standard, provider-agnostic interfaces rather than anything specific to the local environment they were demonstrated on. The same Dockerfile produces the same image on any Docker host; the same three Kubernetes manifests apply unmodified to a managed cluster; and the same Jenkinsfile stages apply regardless of which Docker registry or Kubernetes cluster the gated Push/Deploy stages eventually point at.

8.4 Gating Risk Behind Explicit Decisions

Both the Jenkins pipeline (via its `PUSH_TO_REGISTRY / DEPLOY_TO_K8S` parameters) and the Kubernetes HPA (via its min/max replica bounds) separate "what happens automatically" from "what requires an explicit decision or threshold to be crossed." Tests and builds run on every commit automatically; pushing an image and rolling out a deployment

require an explicit flag; scaling up and down happens automatically, but only between bounds that were explicitly configured in advance.

8.5 Preserving History Rather Than Overwriting It

The Airflow batch job upserts into `predictions.csv` rather than replacing it on every run, so the record of predictions accumulates over time rather than only reflecting the most recent run. MLflow's run store behaves the same way at the model-training level — every run that has ever been logged remains queryable, rather than only the most recent one being visible.

9. Project Structure

The components described in this document are organized on disk as follows, reflecting the file and folder paths referenced throughout the earlier sections:

```
Productionization_flight_price_prediction/
├── app.py          # Flask entry point (home route, form handling)
├── predict.py      # preprocess(), predict(), batch_predictions()
├── pkl_files/
│   ├── encoder.pkl    # fitted OneHotEncoder
│   ├── scaler.pkl     # fitted MinMaxScaler
│   └── best_xgb_model.pkl # tuned XGBoost regressor
├── templates/      # HTML form + result template
├── load_distance_time.csv # per-route avg time/distance lookup
├── predictions.csv  # accumulating batch-scored predictions
├── requirements.txt
├── Dockerfile
├── Jenkinsfile
├── deployment.yaml / service.yaml / hpa.yaml # Kubernetes manifests
├── tests/
│   └── test_predict.py # 2 pytest smoke tests
├── infra/
│   ├── airflow/      # Docker Compose: webserver, scheduler, DB
│   └── dags/
│       └── flight_price_batch_predictions.py
```

Separately, the hotel recommendation Streamlit app and its own saved artifacts (`collaborative_recommendation.pkl` and the scaled content-feature matrix produced in `Hotel_Recommendation.ipynb`) form an independent, self-contained project, since it has no dependency on the flight-price serving path described above.

Finally, `mlruns/` — MLflow's local tracking store — sits alongside `flight_ticket_price_prediction.py`, the training script that produces every run described in Section 7, and is read by the MLflow tracking server to power the UI shown throughout that section.

10. Summary — What Has Been Built

Taken together, the seven components documented above form a complete, working MLOps loop around the flight price prediction model, plus a fully separate interactive application around the hotel recommendation model:

- A tuned XGBoost regression model, selected and tuned through a tracked, four-run MLflow comparison.
- A Flask REST API that serves that model in real time, reusing the exact preprocessing pipeline built during training.
- A Docker image that packages the Flask app for identical behavior across any host.
- A Kubernetes deployment running that image as 3 autoscaling replicas behind a stable load-balanced service, verified against a local minikube cluster using manifests portable to a managed cloud cluster.
- An Airflow DAG that runs the same prediction logic in scheduled batches, building up a continuously growing, deduplicated record of predicted prices.
- A Jenkins pipeline that tests every change automatically and gates the riskier push/deploy actions behind explicit parameters.
- A hybrid hotel recommendation system — combining SVD-based collaborative filtering with content-based cosine similarity, including a cold-start fallback — exposed through a three-panel Streamlit application for recommendations, exploration, and booking-history review.

Every piece described in this document was run and verified directly rather than only configured on paper. The table below summarizes exactly what was checked for each component and where the evidence for it appears in this document:

Component	Verification Performed	Evidence
Flask API	Form submitted locally; predicted price rendered correctly for a real route.	Figures 1.1–1.2
Streamlit App	All three panels (recommend, explore, booking history) exercised against real user codes.	Figures 2.1–2.3
Docker	Image built from a clean cache; containerized app produced an identical prediction to the local run.	Figures 3.1–3.4
Kubernetes	Manifests applied to minikube; 3 replicas and autoscaler observed running against the built image.	Figure 4.1
Airflow	DAG triggered both manually and on schedule; run history shows consecutive successful executions.	Figures 5.1–5.2

Jenkins	Pipeline run end to end with both smoke tests passing; steps inspected stage by stage.	Figures 6.1–6.2
MLflow	Four training runs compared side by side; winning run's full metric set inspected and confirmed registered.	Figures 7.1–7.2

This constitutes a full, reproducible path from a trained model to a served, containerized, orchestrated, tested, and tracked production system — every stage between "model exists" and "model is running, scaling, scheduled, tested, and tracked" has a working implementation and a verified result behind it.